Anti-ProGuard: Towards Automated Deobfuscation of Android Apps

Richard Baumann
University Erlangen-Nuremberg, Germany
richard.baumann@fau.de

Mykolai Protsenko
Fraunhofer AISEC, Garching near Munich, Germany
protsenko@aisec.fraunhofer.de

Tilo Müller
Friedrich-Alexander University Erlangen-Nuremberg, Germany
tilo.mueller@cs.fau.de

ABSTRACT

A wide adoption of obfuscation techniques by Android application developers, and especially malware authors, introduces a high degree of complication into the process of reverse engineering, analysis, and security evaluation of third-party and potentially harmful apps.

In this paper we present the early results of our research aiming to provide reliable means for automated deobfuscation of Android apps. According to the underlying approach, deobfuscation of a given app is performed by matching its code parts to the unobfuscated code stored in a database. For this purpose we apply well-known software similarity algorithms, such as SimHash and n-gram based ones. As a source of unobfuscated code can serve open source apps and libraries, as well as previously analyzed and manually deobfuscated code.

Although the presented techniques are generic in their nature, our current prototype mainly targets Proguard, as one of the most widely used protection tools for Android performing primarily renaming obfuscation. The evaluation of the presented Anti-ProGuard tool witnesses its effectiveness for the considered task and supports the feasibility of the proposed approach.

CCS CONCEPTS

• **Security and privacy** → **Software reverse engineering**; Malware and its mitigation; • **Information systems** → *Similarity measures*;

KEYWORDS

Android, Reverse Engineering, Deobfuscation, Software Similarity

ACM Reference format:
Richard Baumann, Mykolai Protsenko, and Tilo Müller. 2017. Anti-ProGuard: Towards Automated Deobfuscation of Android Apps. In *Proceedings of SHCIS'17, Neuchâtel, Switzerland, June 21-22, 2017,* 6 pages.
DOI: 10.1145/3099012.3099020

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SHCIS'17, Neuchâtel, Switzerland
© 2017 ACM. 978-1-4503-5271-0/17/06…$15.00
DOI: 10.1145/3099012.3099020

1 INTRODUCTION

In November of 2007 Google announced their operating system for mobile devices called Android. A few months later, in September of 2008, the first Android smartphone was introduced by T-Mobile.

Currently, for several years in a row, Android keeps a clear leadership among the operating systems in the smartphone market. According to the global marketing intelligence company IDC, the market share of Android has amounted to 87.6% in the second quarter of 2016. The number of apps available to Android users in the official Google Play app store alone has reached the 2.4 million mark in September 2016 [4]. Together with the growing popularity and the number of various apps, the malware threat increases as well. The Mobile App Reputation Service by Trend Micro has counted 16.6 million malware detections as of August 2016, which is a 40% leap from January [3]. Given this scale of the problem, a reliable detection and both manual and automated analysis becomes a more and more cumbersome task. The required effort is additionally increased by the broad application of obfuscation transformations.

In this paper we propose a novel approach aiming to assist analysis and reverse engineering by providing the means for automated deobfuscation of Android apps. In a nutshell, our approach utilizes software similarity algorithms to identify matches of the code parts from a given app in a database of already known code, such as open source third party libraries. As a result, an unobfuscated version of the code can be found thus evading the need of a repeated analysis and reverse engineering, saving time and effort for the previously unknown code parts and increasing the overall productivity of the process. The presented method is generic, such that in general it may be used against various obfuscation transformations with various granularity, depending on the utilized software similarity algorithms. Here we present our first results, which are represented by the Anti-ProGuard tool, as the name suggests targeting Proguard. The Proguard obfuscation tool, as the part of the Android SDK, certainly belongs to the most widely used ones for Android. Aside of code optimization and shrinking it applies a single protection technique, which is identifier renaming.

The task of Anti-ProGuard is to automatically find code parts in its database, which are similar to the code contained in the app at hand, and use the established similarity relationship to suggest meaningful names to its packages, classes, and methods. As an output, our tool produces a deobfuscated `.apk` file, which then can be processed by the tools of the analyst's choice, such as signature-based malware detection means or decompilers.

An additional use case of Anti-ProGuard is defined by the possibility of sharing work results and findings between multiple analysts or researchers: The data of previously processed apps or their

parts can be stored in the central database and used afterwards by the community members.

The practical evaluation of Anti-ProGuard has indicated its effectiveness in the deobfuscation of apps protected by Proguard, and thus the feasibility of the proposed approach in general.

1.1 Related Work

Next we provide a review of the previous research results related to the scope of our work, such as software similarity, plagiarism detection, and deobfuscation.

A system for source code similarity measurement was proposed by Đurić and Gašević [26], which utilizes the Running-Karp-Rabin Greedy-String-Tiling (RKR-GST) algorithm and the winnowing algorithm. In short, the RKR-GST algorithm is based on substring matches between the given strings, and the winnowing algorithm generates n-grams of the input string and hashes them, creating a sequence of hashes.

IDA Pro, a well known reverse engineering framework provides a set of library signatures called F.L.I.R.T. [8], aiming to detect third-party libraries in binary executables.

Barrón-Cedeño and Rosso [11] introduced an approach addressing plagiarism detection for text documents. For this purpose, they use an n-gram based technique. Their evaluation suggests the best performance being achieved using $n = 2$ or $n = 3$.

Choi et al. [18] propose a similarity measure for binary executables based on the patterns of Windows API calls. Their static birthmark extraction technique is implemented on top of IDAPro.

For Android, a third-party library detection tool was proposed by Ma et al. [24]. The main feature they used for detection is the frequency of the Android API calls.

Chen et al. [15] propose a precise and scalable repackaged malware detection method for Android, which utilizes similarity measurement of code and UI structure of apps. Their approach is based on comparison of a newly submitted app to those already available in the app store. Identifying the difference in code between two apps sharing a similar UI structure, they assume one of the apps to be a repackaged version of the other. Similarity of both UI structure and code is computed with the same approach, that represents a graph as a three-dimensional structure, the mass center of which can be considered as its unique fingerprint. In case of the code comparison, the underlying graph is the control flow graph of the method.

Bichsel et al. [12] have developed a deobfuscation tool for Android apps. Their approach is based on the deep learning from a large amount of unobfuscated apps, which they refer to as "Big Code". Their methodology was found to be effective, being able to restore the most of the identifier names obfuscated by Proguard.

Other related publications include Yadegari [27], Keivanloo et al. [23], Chilowicz et al. [17], Chen et al. [16], Chen et al. [14], Charikar [13].

1.2 Contributions

In summary, with this paper we make the following contributions.

- We propose to our best knowledge the first generic approach to automated deobfuscation based on software similarity algorithms.
- As a proof of concept, we present a prototype implementation of the Anti-ProGuard tool, which targets identifier renaming obfuscation applied by Proguard, one of the most widely used protection tools for Android.
- We perform a practical evaluation of Anti-ProGuard demonstrating the feasibility of the discussed approach.

1.3 Outline

The rest of the paper is organized as follows. Section 2 introduces background information about the topics closely related to the scope of our work. Section 3 is devoted to the Anti-ProGuard tool and describes its design and implementation. The practical evaluation of this tool is presented in Section 4. We discuss the obtained results in Section 5. Finally, we conclude this paper with Section 6.

2 BACKGROUND

In this section we provide an overview of the background information concerning the Android system, obfuscation, and the tools used in our work.

2.1 Android

Android is a software stack built upon Linux kernel. It can be described as a stack that includes the operating system itself, some middleware and a few key applications [20]. The Android stack is divided into, from bottom to top: Linux kernel, libraries and Android Runtime, application framework, and applications.

The Linux kernel contains most of the drivers, like the display, camera and audio drivers. The libraries part of the stack includes C and C++ libraries, like `SSL`, `libc`, and `WebKit`. Providing those as C/C++ libraries allows for faster execution of applications. The Android Runtime consists of additional libraries and the Runtime itself. The runtime is represented by ART, which replaced the Dalvik VM a while ago [6]. On top of the libraries and the Android runtime, the application framework is placed. It consists of framework functionality like the activity manager, the window manager, services for locations and resources. On top of that, on the top end of the stack, the applications run. They usually access the lower parts of the stack to get access to hardware, like the camera, the screen, or the GPS signals.

Common apps are those for calling, web browsing, SMS services, instant messengers, and email. In addition to the pre-installed apps, most end user devices allow for additional installations of applications.

Android apps are usually developed in the Java language, also providing the possibility of using native code available through the Java Native Interface (JNI). The Java part of an app is compiled into Dalvik bytecode, which originally targeted the register-based Dalvik virtual machine.

The compiled code and all additional resources, like images, sound, `AndroidManifest.xml`, and other `.xml` files are bundled into a zip file that has the file extension `.apk`. The AndroidManifest includes important meta information about the app, especially the permissions the app requires.

For the scope of our work, the highest importance has the bytecode contained within the application packages, contained in the

`classes.dex` file. There may be additional classes files that are then called `classes2.dex`, `classes3.dex` and so on [7].

2.2 Obfuscation

According to Collberg and Thomborson [19], an obfuscating transformation can be defined as follows.

Definition 1. *Given a set of obfuscating transformations $T = \{T_1, ..., T_n\}$ and a program P consisting of source code objects (classes, methods, statements, etc.) $\{S_1, ..., S_k\}$, find a new program $P' = \{..., S'_j = T_i(S_j), ...\}$ such that:*

- *P' has the same observable behavior as P, i.e., the transformations are **semantics-preserving**.*
- *The **obscurity** of P' maximized, i.e., understanding and reverse engineering P' will be strictly more time-consuming than understanding and reverse engineering P.*
- *The **resilience** of each transformation $T_i(S_j)$ is maximized, i.e., it will be either difficult to construct an automatic tool to undo the transformations or executing such a tool will be extremely time-consuming.*
- *The **stealth** of each transformation $T_i(S_j)$ is maximized, i.e., the statistical properties of S'_j are similar to those of S_j.*
- *The **cost** (the execution time/space penalty incurred by the transformations) of P' is minimized.*

It shows that the basic idea behind obfuscation is to transform a program P into a program P' that takes longer to reverse-engineer while having the same observable behavior. There are multiple uses of obfuscation: Preventing theft of intellectual property (e.g., modern algorithms or coding techniques), or hiding malicious code from detection tools. The most commonly used obfuscation techniques include identifier renaming, control flow obfuscation, and data transformations.

For Android, ProGuard is one of the many obfuscation tools available [1], which additionally has optimizing, shrinking, and preverifying capabilities. To elaborate, it detects and removes unused classes and their methods and attributes, optimizes the bytecode and renames packages, classes, fields and methods to short and meaningless names. Being a part of the Android SDK, Proguard can be expected to be the most widely adopted obfuscation tool for Android.

2.3 Smali and APKTool

Smali/Baksmali is an assembler/disassembler for the dex file format used by Dalvik, providing a human readable representation of Dalvik bytecode [9]. Baksmali is also utilized by Apktool [5], which is a reverse engineering tool for Android.

Aside of disassembling the app's bytecode with Baksmali, Apktool extracts the resources from the `.apk` file, and decodes the `.xml` files from the binary format. Furthermore, it provides the possibility to build a fully functional app back from its decoded components, incorporating all the changes introduced into resources and bytecode disassembly. This allows for both automated and manual modification of `.apk` files.

3 ANTI-PROGUARD: IMPLEMENTATION

This section describes the design and prototype implementation of the Anti-ProGuard tool. Anti-ProGuard is implemented in python3, a high-level and general purpose scripting language. The processing of the `.apk` files is facilitated by APKTool, which is able to decompose an application package in its parts and disassemble the bytecode using Smali.

The implementation of Anti-ProGuard is divided into three major parts:

- *Tree analysis*, where the file tree is analyzed and memorized as a tree of objects.
- *Comparison*, where the different methods, classes and packages are compared to their counterparts in the database.
- *Renaming*, where the results of the previous step are used to perform renaming according to the data stored in the database.

Additionally, some simple pre- and post-processing steps are required, such as decomposing and re-building of `.apk` files. Moreover, the database of known code is expected to be filled with a possibly large amount data.

3.1 Tree Analysis

The preprocessing step performed with APKTool generates a `smali` folder for each of the `.dex` files contained in the app, with subfolders corresponding to the package hierarchy of the app's classes. Within these subfolders, each `.smali` file represents a definition of a single class.

In the first step a tree structure reflecting the folder layout of smali files is created. This tree structure is used in the next step to perform comparison of the code base and renaming operations. The parsing of the `.smali` files is performed using regular expressions, extracting such important information as method definitions with the number of parameters, method call sites, class definitions and uses.

Additionally, in the tree analysis step Anti-ProGuard attempts to automatically detect name obfuscation of packages and classes, based on a simplified assumption that Proguard generates short meaningless names in its default configuration. Thus, we have empirically developed a rule of thumb which considers a package to be obfuscated if 82.5% of its classes have names shorter than five characters. Obviously, this approach would not provide correct result in case a custom name obfuscation scheme is used, for instance supplying Proguard with a user-defined dictionary for renaming. For this purpose, Anti-ProGuard allows for the manual identification of obfuscated packages. For each obfuscated package, the tool will then start the deobfuscation process. The unobfuscated parts of code, on the other hand, can be added to the database of known code.

3.2 Comparison

The deobfuscation of packages is facilitated by the pairwise comparison of method fingerprints extracted from the obfuscated code of a given app and method fingerprints from the known packages stored in the database. The comparison step is divided into a bottom-up and a top-down iteration. The bottom-up iteration is executed first, aiming to match considered packages to the known ones. Then,

the top-down iteration matches classes and methods within the previously established package.

3.2.1 Bottom-up Iteration. The methods are processed in the order defined by their length, from large to small. This order has been chosen because it provided faster results, since larger methods have a higher chance to differ from other methods. Note that abstract methods, constructors, and very short methods are excluded from the comparison process.

As another heuristic aiming to increase performance, the most likely matches identified on previous file comparisons are used as 'hints' for the next ones. After comparing a single method, the results of this comparison are used as hints for future comparisons, which then produce new hints for new comparisons and so on.

The actual comparison of methods is performed in several steps, starting with the most lightweight and least precise techniques moving towards more costly and reliable ones, aiming to filter out wrong matches early on in the process thus reducing the overhead. Since Proguard does not change the parameter count of methods, Anti-ProGuard first discards all methods that have a wrong parameter count. In addition to that, methods that are too long or too short to be similar will be not considered as well. For this, a 10% margin in both directions was used.

After these simple filters, the SimHash-based and n-gram based comparisons are performed, which provide more conclusive data about method similarity.

3.2.2 Simhash. The Simhash is an algorithm implementing Locality-Sensitive Hashing (LSH), meaning that two similar features being hashed result in two similar hashes, thus being able to identify not only identical features, but also similar ones. In our work we adapt the approaches presented by Charikar [13] and Manku et al. [25] to similarity measurement of bytecode instructions. As a result, the following algorithm is applied.

- Break the method into instructions.
- Pick a hash size, in our case 256 bit length was selected.
- Create a 256-dimensional vector V, and initialize each dimension with 0.
- Hash each instruction with a normal hash algorithm with the same hash size.
- If the i-th bit of the hash value is 1, increment the i-th component of V by 1. If the i-th value of the hash is 0, then decrement the i-th component of V by 1.
- After all features have been processed, for each dimension i, determine the i-th bit of the result hash with the sign of the i-th dimension of the vector. So if $V[i] > 0$, set $hash[i] = 1$, else $hash[i] = 0$.

With this algorithm we compute three hash variations. The first one takes all instructions in the method, including their arguments, annotations, and labels. The second one skips all labels and annotations considering only bytecode instructions including their arguments. The third hash only takes into account the bytecode instructions, without arguments.

This similarity between the corresponding hashes is calculated using the hamming distance. The hamming distance between two hashes, represented by the bit vectors a and b is defined as the number of positions, i.e. bits, in which a and b differ [22]. For instance, for the bit vectors 0110 and 0111, the hamming distance is 1.

Then the similarity value between hashes a and b can be computed as [25]:

$$sim(a, b) = \frac{length(a) - hamming_distance(a, b)}{length(a)}$$

Such similarity score is computed for all three hash variants described above. If the maximum of these three values is higher than 90%, the comparison advances to the next step, namely the n-gram based similarity test.

3.2.3 N-grams. An n-gram of a feature sequence is defined as a contiguous subsequence of length n [21]. In general, a sequence can be splitted into n-grams with two distinct approaches: an intersecting one and a non-intersecting one. In the non-intersecting approach, each consequent n-gram starts with the feature next to the end of the previous n-gram, such that each feature is covered by exactly one n-gram. In the intersecting approach, the consequent n-gram begins with the feature next to the previous n-gram's start. In our work we use intersecting n-grams, as our empirical investigation suggested their better performance.

Also during development, different values of n have been tested. Following Barrón-Cedeño and Rosso [11], low values for n have been used first, so the tests were done with values between two and four inclusive. After testing, using three as the value for n was providing the most successful results, so it was chosen for the rest of the implementation.

To perform n-gram based similarity check for two methods, each method is represented as a set of intersecting n-grams of their instruction sequences. Then, the similarity score for those two methods is defined as a symmetric difference between those sets:

$$sym_diff(s_1, s_2) = (s_1 \setminus s_2) \cup (s_2 \setminus s_1)$$

where s_1 and s_2 denote the n-gram sets of the first and the second method, respectively. If the amount of n-grams in the result set is less than 20% of the method length, the two methods are considered similar.

Note that Anti-ProGuard allows for the user to manually decide whether a package was identified correctly. To do this, it lists up the different packages that might be the one, in order from highest probability to lowest.

3.2.4 Top-down Iteration. After the package mappings have been established, the top-down iteration is performed. Anti-ProGuard uses the now known packages to try to improve the results for the files and methods within the package. It iterates over the packages top-down and tries to match methods and files to the methods and files found in the database.

This way, each file is mapped to a list of possible files, each with its own chance to be correct. These lists are then sorted such that the results with the highest chance appear first. Iterating over this sorted list, the matching is built. After the files have been matched, the methods contained in each of them are matched as well.

3.3 Renaming

Renaming is done using the results provided from the previous step in a bottom-up manner, starting with methods, then classes,

then packages. Anti-ProGuard prints out a log file that contains all changes done to the apk to help reverse engineers when trying to further analyze the app.

3.3.1 Methods. Method names have to be replaced both in their definitions and the corresponding method call statements. This means, when renaming, one has to rename the method definition in the class that the method is declared in and all calls to this method contained in all packages of the application.

3.3.2 Classes. Renaming classes again includes multiple steps: The file has to be renamed, the class name in the file has to be changed, and all method calls or other accesses to the members of this class have to be updated.

3.3.3 Packages. The package renaming is performed in a similar way to renaming of classes and methods. First of all, the accesses to classes and their members, which belong to the target package have to be updated. In addition to that, the folder structure should be renamed.

3.4 Reassembly

When Anti-ProGuard has performed the three major steps, namely the tree analysis, comparison, and renaming, the Android app package has to be built again using APKTool. Note that the `.apk` file produced by this step may not be runnable. This can occur due to the following reasons. First, Anti-ProGuard may distort the original object-oriented design relationships of the renamed classes, such as to their interfaces or superclasses, the overloaded methods of which might have been a subject of renaming as well. And second, renaming invalidates reflective accesses to the classes and their members.

3.5 Setting up the Database

Before Anti-Proguard can work properly, a database has to be set up. Anti-ProGuard utilizes SQLAlchemy [10] to offer support of multiple database engines, one of them has to be chosen and a fitting database server has to be installed. Then the correct tables have to be created in the database, which can be done automatically by executing a dedicated initialization script. After this step, the database has to be filled with initial data. This can be achieved by importing the database content of a filled database or by letting Anti-ProGuard analyze unobfuscated `.apk`, `.jar`, or `.dex` files. Anti-ProGuard will then generate the n-grams, simhashes, and some additional meta data of those packages and add them to the database.

4 EVALUATION

To evaluate Anti-ProGuard, the database has been initialized and multiple, semi-randomly chosen apps have been deobfuscated. The results of the deobfuscation process have been compared to their unobfuscated counterparts to test their correctness.

4.1 Setup

Next we describe the details of the evaluation setup, such as the utilized hardware and a test set of Android apps.

4.1.1 Hardware. The evaluation tests were performed on two machines, configuration of which is provided below. The first machine is a PC running Windows 10, with an Intel Core i5-4690K at 3.50 GHz and 16.0 GB of RAM. Anti-ProGuard was set up to use MySQL as its backend database and this database was running inside the Linux Subsystem for Windows.

The second machine is a PC running Arch Linux, with an AMD Phenom 9550 Quad-Core CPU at 1.1 GHz and 4.0 GB of RAM. On this machine, Anti-ProGuard has been set up using MariaDB.

4.1.2 Database. For the purpose of our evaluation, the database was initialized with a set of well-known and widely used libraries, such as `android.support.` with sub-packages `v4, v7, v8, v13`, `com.android.vending`, and `org.bouncycastle`.

The selection of popular libraries was facilitated by webpages like appbrain [2], which provide statistics about the usage of libraries in the Google Play Store. Note that many of the libraries have multiple major versions. Anti-ProGuard provides support for adding multiple versions of the same library to the database.

4.1.3 Test Apps. The test apps were semi-randomly selected from the open source repository F-Droid[1], satisfying the following criteria.

- Libraries used: Apps that contained at least a few libraries that had been added to the database prior to the test have been chosen in preference to others.
- Proguard: Apps that made it easy to enable / disable Proguard obfuscation have been chosen in preference.
- Ease of Compilation: Only the apps providing a working gradle scripts for compilation were considered.

The resulting test set contained the following apps:

- Akhyou![2]
- Conversations 1.13.4[3]
- Critical Maps[4]
- Seafile[5]

These apps were compiled both with and without Proguard enabled.

4.2 Results

In summary, Anti-ProGuard was able to correctly identify over 50% of packages in the evaluated apps. The main limitation, preventing our tool from detecting other packages, was found to be packages with a dominance of interfaces, abstract classes, and annotations, as well as those having mostly simple data types. Leaving these packages out of calculation rises the detection rate to about 80%.

The reason for the lower accuracy on packages with less code lies in the fact that our approach requires a certain amount of characteristic code to draw reasonable conclusions about the code similarity. However, one can argue that classes and packages with

[1] https://f-droid.org/
[2] https://f-droid.org/repository/browse/?fdfilter=Akhyou&fdid=dulleh.akhyou.fdroid
[3] https://f-droid.org/repository/browse/?fdfilter=Conversations&fdid=eu.siacs.conversations
[4] https://f-droid.org/repository/browse/?fdfilter=Critical+Maps&fdid=de.stephanlindauer.criticalmaps
[5] https://f-droid.org/repository/browse/?fdfilter=Seadroid&fdid=com.seafile.seadroid2

higher concentration of unique code pose as more interesting targets for analysis and thus a better performance on such code parts compensates for the weaker performance on packages containing, for instance, mostly interfaces and abstract classes.

An additional shortcoming of the current prototype was found to be a comparatively high processing time, which ranged from four to six hours for the evaluated apps. Nevertheless, such time overhead can be surely considered more than tolerable, as the deobfuscation can be performed automatically without human interaction and saves the effort during the reverse engineering. The identification of the main performance bottlenecks and their optimization is a subject of the future work.

5 DISCUSSION

The evaluation of Anti-ProGuard, the prototype implementation of the approach we propose in this paper, described in the previous section provided some optimistic data regarding the high potential of the method, which is able to correctly rename the majority of packages for the apps which participated in the tests. At the same time the evaluation also suggested a room for improvement, demonstrating the shortcomings of the current version. Precisely, both the accuracy of the deobfuscation and runtime performance of the tool may be increased.

To address these issues, further research and investigation is required, taking into consideration a variety of software similarity algorithms and their possible combination. In particular, a centroid-based approach proposed by Chen et al. [15], seems to provide a promising alternative the techniques used in our current work.

6 CONCLUSION

In this paper we presented our preliminary research results, setting an ambitious goal of automated deobfuscation of Android apps. The proposed approach makes use of software similarity algorithms to find code samples similar to the code being analyzed. We present a prototype implementation of Anti-ProGuard, a tool applying the aforementioned technique against the renaming obfuscation provided by Proguard, one of the most popular protection tools for Android.

In the practical evaluation Anti-ProGuard demonstrated its effectiveness by successfully identifying over 50% of known packages, and hence providing evidence for the feasibility of the approach.

In our future research, we plan to experiment with a larger number of similarity algorithms aiming to increase the quality of deobfuscation, evaluate its resilience and applicability against obfuscation transformations other than renaming, and attempt to increase the runtime performance of the tool.

ACKNOWLEDGMENTS

The research leading to these results was supported by the "Bavarian State Ministry of Education, Science and the Arts" as part of the FORSEC research association.

REFERENCES

[1] ProGuard. https://stuff.mit.edu/afs/sipb/project/android/sdk/android-sdk-linux/tools/proguard/docs/index.html. (2011). Access: October 8th, 2016.
[2] Appbrain - Android library statistics. http://www.appbrain.com/stats/libraries/. (1 November 2016). Access: November 2nd, 2016.
[3] DressCode and its Potential Impact for Enterprises. http://blog.trendmicro.com/trendlabs-security-intelligence/dresscode-potential-impact-enterprises/. (29 September 2016). Access: October 3rd, 2016.
[4] Number of available applications in the Google Play Store from December 2009 to September 2016. https://www.statista.com/statistics/266210/number-of-available-applications-in-the-google-play-store/. (2016). Access: October 3rd, 2016.
[5] Apktool - A tool for reverse engineering Android apk files. https://ibotpeaches.github.io/Apktool/. (n.d.). Access: October 10th, 2016.
[6] ART and Dalvik. https://source.android.com/devices/tech/dalvik/. (n.d.). Access: October 5th, 2016.
[7] Configure Apps with Over 64K Methods. https://developer.android.com/studio/build/multidex.html. (n.d.). Access: November 8th, 2016.
[8] IDA F.L.I.R.T. Technology: In-Depth. https://www.hex-rays.com/products/ida/tech/flirt/in_depth.shtml. (n.d.). Access: October 3rd, 2016.
[9] Smali. https://github.com/JesusFreke/smali. (n.d.). Access: October 10th, 2016.
[10] SQLAlchemy - Dialects. http://docs.sqlalchemy.org/en/latest/dialects/index.html. (n.d.). Access: October 17th, 2016.
[11] Alberto Barrón-Cedeño and Paolo Rosso. 2009. On automatic plagiarism detection based on n-grams comparison. In *European Conference on Information Retrieval*. Springer, 696–700.
[12] Benjamin Bichsel, Veselin Raychev, Petar Tsankov, and Martin Vechev. 2016. Statistical Deobfuscation of Android Applications. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 343–355.
[13] Moses S Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *Proceedings of the thiry-fourth annual ACM symposium on Theory of computing*. ACM, 380–388.
[14] Kai Chen, Peng Liu, and Yingjun Zhang. 2014. Achieving accuracy and scalability simultaneously in detecting application clones on android markets. In *Proceedings of the 36th International Conference on Software Engineering*. ACM, 175–186.
[15] Kai Chen, Peng Wang, Yeonjoon Lee, XiaoFeng Wang, Nan Zhang, Heqing Huang, Wei Zou, and Peng Liu. 2015. Finding unknown malice in 10 seconds: Mass vetting for new threats at the google-play scale. In *24th USENIX Security Symposium (USENIX Security 15)*. 659–674.
[16] Kai Chen, Xueqiang Wang, Yi Chen, Peng Wang, Yeonjoon Lee, XiaoFeng Wang, Bin Ma, Aohui Wang, Yingjun Zhang, and Wei Zou. 2016. Following DevilâĂŹs Footprints: Cross-Platform Analysis of Potentially Harmful Libraries on Android and iOS. (2016).
[17] Michel Chilowicz, Etienne Duris, and Gilles Roussel. n.d.. Syntax tree fingerprinting: a foundation for source code similarity detection. *Universitye Paris-Est* (n.d.).
[18] Seokwoo Choi, Heewan Park, Hyun-il Lim, and Taisook Han. 2007. A static birthmark of binary executables based on API call structure. In *Annual Asian Computing Science Conference*. Springer, 2–16.
[19] Christian S. Collberg and Clark Thomborson. 2002. Watermarking, tamper-proofing, and obfuscation-tools for software protection. *IEEE Transactions on software engineering* 28, 8 (2002), 735–746.
[20] Android Developers. What is android. (2011).
[21] Georgia Frantzeskou, Efstathios Stamatatos, Stefanos Gritzalis, Carole E. Chaski, and Blake Stephen Howald. 2007. Identifying Authorship by Byte-Level N-Grams: The Source Code Author Profile (SCAP) Method. *IJDE* 6, 1 (2007).
[22] Richard W Hamming. 1950. Error detecting and error correcting codes. *Bell System technical journal* 29, 2 (1950), 147–160.
[23] Iman Keivanloo, Chanchal K Roy, and Juergen Rilling. 2012. Java bytecode clone detection via relaxation on code fingerprint and semantic web reasoning. In *Proceedings of the 6th International Workshop on Software Clones*. IEEE Press, 36–42.
[24] Ziang Ma, Haoyu Wang, Yao Guo, and Xiangqun Chen. 2016. LibRadar: fast and accurate detection of third-party libraries in Android apps. In *Proceedings of the 38th International Conference on Software Engineering Companion*. ACM, 653–656.
[25] Gurmeet Singh Manku, Arvind Jain, and Anish Das Sarma. 2007. Detecting near-duplicates for web crawling. In *Proceedings of the 16th international conference on World Wide Web*. ACM, 141–150.
[26] Zoran Đurić and Dragan Gašević. 2012. A source code similarity system for plagiarism detection. *Comput. J.* (2012), bxs018.
[27] Babak Yadegari. 2016. Automatic Deobfuscation and Reverse Engineering of Obfuscated Code. (2016).